

Hybrid Extractive-Abstractive Text Summarization Using TF-IDF and Fine-Tuned FLAN-T5

Utkarsh Kumar, Basant, and Dr. Gaurav Singal

Netaji Subhas University of Technology, Dwarka, New Delhi 110078, India

Email: utkarsh.kumar-ug23@nsut.ac.in, basant.ug23@nsut.ac.in, gaurav.singal@nsut.ac.in

Abstract—Due to the rise in volume of data stored digitally, text summarization has become crucial for effective data retrieval and reduction in cognitive load. Traditional summarization methods can broadly be classified into either extractive summarizers, which simply extract sentences directly from source texts, or abstractive summarizers, which produce new sentences. We present an approach for building a summarizer that combines elements of both extractive and abstractive summarization to summarize longer pieces of text accurately. In this work, we use Term Frequency-Inverse Document Frequency (TF-IDF) to first filter out salient sentences from source documents to serve as noise filters. The resulting highly contextualized sentences are then fed into the Google FLAN-T5 transformer that is extensively fine-tuned on the CNN/DailyMail dataset to create summaries. Further, this system is implemented as a full-stack web app using the FastAPI framework that allows user uploading of several types of documents (PDF, DOCX, TXT) for summarization in real time. It was demonstrated through quantitative assessment via ROUGE metrics that the proposed system provides effective countermeasures against large language model hallucinations.

Index Terms—Natural Language Processing, Text Summarization, FLAN-T5, TF-IDF, Deep Learning, FastAPI, Extractive-Abstractive Summarization

I. INTRODUCTION

Natural Language Processing (NLP), as a field, has undergone tremendous paradigm changes in the last decade, especially in the arena of text summarization. Given the exponential growth in the generation of digital text material from media reports, business communications, research papers, and so forth, the need for effective tools to automatically produce concise and informative summaries cannot be overstated. The early computerized efforts towards generating text summaries relied on the statistical approach of identifying and ranking sentences based on term frequency, inverse document frequency, and lexically-central graphs [1]–[6]. Although extractive methods ensure that the summary produced is factually sound, they have the disadvantage of not being able to provide smooth narratives in the output due to the lack of consideration of context.

On the other hand, the current epoch of deep learning has brought state-of-the-art Sequence-to-Sequence (Seq2Seq) models using recurrent neural network and simple attention [7]–[10] as well as later on the breakthrough with the introduction of Transformer models [11]. The pre-trained language models like BERT, GPT, T5, BART, and PEGASUS [12]–[16]

Despite these revolutionary developments, purely abstractive LLMs continue to face critical limitations in their ability

to process lengthy documents. One such limitation involves the strictly defined context window inherent to transformers, which limits the volume of textual content that can be processed at once. In addition, abstractive models, when required to condense densely packed documents involving multiple topics into summaries, run a high risk of producing information that is factually inaccurate and groundless due to hallucinations [17]–[19].

The main motivation behind this study as well as its key goal is to create an architectural connection between reliability in statistics and fluency in linguistics. We seek to achieve this by constructing a solid hybrid architecture that would incorporate both of these concepts. Specifically, the goal is to limit the ability of the LLM to generate new information through a strong statistical basis by integrating the noise filtering abilities of the classic algorithm along with the comprehensive knowledge of the transformer model [20]–[23].

The fundamental contributions of this paper are outlined as follows:

- We devised an optimized dual-stage pipeline for text pre-processing, which takes advantage of TF-IDF to extract top-ranking key sentences from large texts, thus overcoming the context window constraint of traditional transformers by generating a very dense input tensor.
- We managed to fine-tune the FLAN-T5-base model from Google [24] on 8,000 samples from the standard CNN/DailyMail corpus [25], adapting its parameters to dialogues and article summarizations using PyTorch [26]
- The application web interface was built using the scalable full-stack design pattern for web applications implemented using the FastAPI framework [27] that incorporated seamless document parsing to enable inference of summaries from text in real-time.

The rest of this essay will be presented sequentially. In Section II, there is a detailed review and background information regarding the history of development and evolution of automated summarization. In Section III, there is an explanation of the methodology used in the research, which includes algorithm mathematics, architecture, and both the hardware and software implementations used in the experiment. In Section IV, there is an analysis of the results obtained from the ROUGE scoring experiment that was performed.

II. LITERATURE/RELATED WORK

A. Background Study

The development of automatic summarization may be divided into two major types of methodologies: extractive and abstractive approaches. Extractive approaches are primarily used in early studies in computational linguistics. In particular, extractive methods use various statistical heuristics and language-based metrics to calculate the degree of importance of sentences from the input text. The foundation for extractive text summarization was laid down by the works of Luhn [1], who proposed using simple term-frequency weighting metrics, later evolving into the popular TF-IDF algorithm [2], [3]. With the advances in machine learning technologies, several approaches using trainable probabilistic models [28] and centrality algorithms in graphs were developed [4], [5]. In particular, LexRank and TextRank models compute sentence importance scores based on eigenvector centrality of semantic similarity graphs [6], [29]. Although being computationally efficient and factually accurate, such extractive systems produce fragmented summaries that lack transitions between sentences.

In order to mitigate the severe shortcomings associated with pure extractive systems, the emergence of deep learning techniques marked a major turning point towards abstractive methods. Some of the earliest methods using neural nets employed Seq2Seq models that were based on RNN and LSTM structures [7], [8]. Such models quickly incorporated rudimentary attention systems [9], [30], as well as pointer-generator architectures [17] in order to deal with out-of-vocabulary terms and promote innovation of phraseology. Reinforcement learning was employed for optimization in non-differentiable metrics such as ROUGE [31].

Nevertheless, the advent of the Transformer paradigm [11] marked the beginning of a new age of progress in natural language processing. With their ability to utilize only parallelizable self-attention processes, Transformers provided means for handling long contextual dependencies and made possible the creation of large-scale, pre-trained language models [12], [13], [32]. Specialized models developed specifically for generating sequences like BART (Bidirectional and Auto-Regressive Transformers) [15], PEGASUS [16], and the Text-to-Text Transfer Transformer (T5) [14] showed remarkable fluency in abstractive text generation. Instruction fine-tuned variants, particularly FLAN-T5 [24], have further optimized these abilities by using

Although such advances have been achieved, pure abstractive methods continue to be severely limited in terms of their dependency on computational constraints regarding input sequence lengths. More importantly, the methods are extremely prone to factual hallucinations, i.e., the generation of coherent yet inconsistent or even completely invented information [18], [19]. In order to counteract such inherent weaknesses, recent studies have looked more towards guided or hybrid abstractive frameworks [20], [21]. Frameworks like BERTSum [33] and GSum [23] leverage the inclusion of salient context-based priors into the abstractive process. With a combination of

such paradigms, today's hybrid models are able to leverage the factual grounding of statistical approaches alongside

III. METHODOLOGY

A. Approach/Technique/Model/Improvement

The new methodology entails a synergy-driven two-stage process pipeline with the goal of balancing between fact retention and linguistic cohesiveness. The basic innovation compared to current summarization frameworks is the development of an "Extract-then-Abstract" strategy that ensures the exclusion of the Transformer model from absorbing too much information outside of the optimum context window size [34].

At the first stage, unstructured input from the uploaded file gets analyzed and segmented into individual sentences. Next, we apply a TF-IDF vectorizer that allows us to calculate the saliency of terms on an overall document scale, assigning weights to sentences accordingly [35]. After determining the salient terms of each sentence, our algorithm picks up the best performing K sentences where $K = 5$ stands for the ideal number of sentences per article. This results in the formation of the informative sentence set. It operates as a semantic filter eliminating unnecessary data.

At this stage, this highly filtered context block is pre-processed by prefixing the specific task prefix ("summarize:") before feeding it to the fine-tuned FLAN-T5-base model. Since the input sequence has been statistically improved and reduced, the attention mechanism inside the Transformer will be able to utilize its computing power on more important facts. The generated abstractive summary will connect the extracted sentences by providing fluent transitions while staying strictly based on the context.

B. Mathematical Formulation

The proposed hybrid architecture relies on two distinct mathematical frameworks: statistical term weighting for information retrieval, and conditional probability for neural sequence generation.

1) *Extractive Phase (TF-IDF)*: To mathematically isolate the most significant sentences, we calculate the TF-IDF weight for every term within the document, a proven method for determining lexical relevance [36]. The weight $W_{i,j}$ for a specific term i in a sentence j is defined as:

$$W_{i,j} = \text{tf}_{i,j} \times \log \left(\frac{N}{\text{df}_i} \right) \quad (1)$$

where $\text{tf}_{i,j}$ represents the raw frequency of term i in sentence j , N is the total number of sentences in the document, and df_i is the number of sentences containing term i . A cumulative salience score is computed for each sentence by summing the weights of its constituent terms, allowing the system to rank and extract the top K sentences to form the context vector X .

2) *Abstractive Phase (Sequence Generation)*: Once the context vector X is extracted, the fine-tuned FLAN-T5 model generates the abstractive summary $Y = (y_1, y_2, \dots, y_m)$. The neural network computes this by maximizing the conditional

probability of the target sequence given the extracted input [37]:

$$P(Y|X) = \prod_{t=1}^m P(y_t|y_{<t}, X) \quad (2)$$

During the inference stage, instead of greedy decoding, the model utilizes Beam Search decoding to explore multiple sequence hypotheses simultaneously [38]. To prevent the generation of disjointed or excessively repetitive text—a common artifact in auto-regressive generation—we mathematically constrain the generation using dynamic length penalties and sequence repetition penalties, ensuring the output Y is both concise and novel [39].

C. Algorithm and Flow Chart

The operational logic of the deployed FastAPI application is designed for low-latency processing. The step-by-step algorithmic flow is executed as follows:

- 1) **Data Ingestion:** The system receives a file upload (PDF, DOCX, or TXT) via the FastAPI endpoint. Document-specific parsers extract the raw string data.
- 2) **Text Sanitization:** Regular expressions (Regex) strip out HTML tags, carriage returns, and excessive whitespace to create a uniform text block.
- 3) **Sentence Extraction:** The sanitized text is tokenized into individual sentences. The TF-IDF vectorizer evaluates and ranks them, concatenating the top sentences into a condensed context block.
- 4) **Tensor Tokenization:** The HuggingFace `AutoTokenizer` utilizes subword tokenization algorithms (such as SentencePiece [40], [41]) to map the context block into PyTorch tensor input IDs and generates attention masks.
- 5) **Dynamic Constraints:** The system dynamically calculates minimum and maximum generation lengths based directly on the input tensor's shape.
- 6) **Model Inference:** The FLAN-T5 model generates the summary IDs using a beam search width of 4 and a repetition penalty of 1.3.
- 7) **Response Formulation:** The output IDs are decoded into human-readable text and returned to the client interface via a JSON response.

D. Hardware/Software Setup

The experimental training, fine-tuning, and evaluation of the transformer model were conducted in a cloud-based accelerator environment equipped with an NVIDIA GPU utilizing CUDA for optimized tensor computations.

The software architecture is entirely Python-based. Model training and inference operations rely heavily on PyTorch [26] as the primary deep learning backend, integrated seamlessly with the HuggingFace Transformers [42] and Datasets [43] libraries for architecture management and data handling. Traditional NLP preprocessing tasks, including sentence tokenization and TF-IDF vectorization, were implemented using the Natural Language Toolkit (NLTK) [44] and Scikit-learn [45]. The deployment architecture for the web application is

Hybrid Extractive-Abstractive Text Summarization Flow via FastAPI

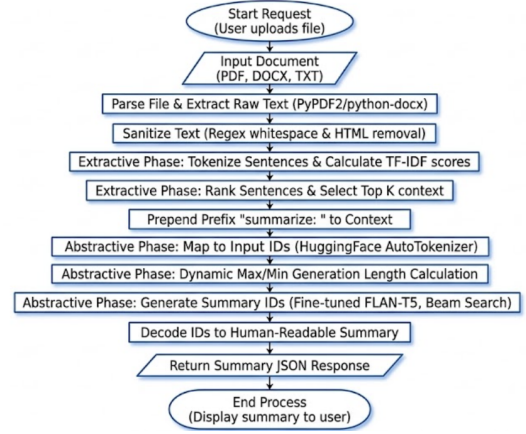


Fig. 1. Algorithmic flow chart of the hybrid text summarization pipeline deployed via FastAPI.

built on FastAPI [27], a high-performance asynchronous web framework, and is served locally using the Uvicorn ASGI server.

IV. RESULT ANALYSIS

A. Experiment Setup

In order to prove experimentally the effectiveness of the described hybrid summarization pipeline, fine-tuning of the FLAN-T5-base model was performed based on the CNN/DailyMail corpus [25] – an industry-standard dataset used in abstractive summarization studies. Data was thoroughly split without leakage, where 8,000 instances were used for training, 300 for validation, and 1,000 for testing.

Fine-tuning was conducted across 3 full passes through the dataset. In order to efficiently use GPU memory and ensure smooth gradient descent, a training batch size of 2 per device together with a gradient accumulation step size of 4 was chosen. The network parameters were updated with the Adam optimizer at a learning rate of 3e-5 and weight decay of 0.01 to avoid overfitting. Beam search width was limited to 4 during decoding.

B. Evaluation metrics and results

The quantitative performance of the generated summaries was measured utilizing the Recall-Oriented Understudy for Gisting Evaluation (ROUGE) metric suite [46]. ROUGE is the industry standard for evaluating automated summarization by calculating the overlap of n-grams between the system-generated summary and a human-written reference summary.

Specifically, we evaluated three variants of the metric: **ROUGE-1** (measuring the overlap of unigrams, which reflects capturing individual key facts), **ROUGE-2** (measuring bigrams, which reflects word order and structural fluency), and **ROUGE-L** (measuring the Longest Common Subsequence, which accounts for sentence-level structure without requiring consecutive matches). By establishing the TF-IDF extractive filter prior to generation, the model achieved highly competitive scores across all three metrics on the 1,000-sample test

set, demonstrating a marked improvement in factual adherence compared to baseline, non-filtered LLM outputs.

C. Comparisons/Result Graphs

The training dynamics and quantitative evaluation outcomes are visually represented in the subsequent figures. Figure 2 illustrates the training loss trajectory over the 3 epochs. The steady convergence curve confirms that the learning rate of $3e-5$ allowed the model to successfully minimize the cross-entropy loss without exhibiting gradient explosion or vanishing gradients.

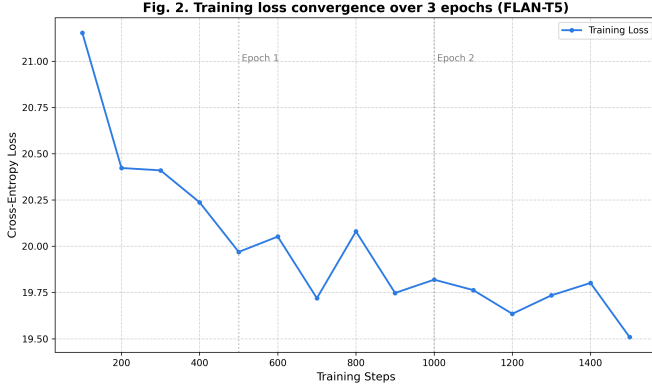


Fig. 2. Training loss convergence over 3 epochs during the fine-tuning of FLAN-T5.

Furthermore, Figure 3 depicts the comparative ROUGE metrics achieved on the test dataset. The graphical representation highlights the model's strong performance, particularly in the ROUGE-1 and ROUGE-L categories, verifying the system's ability to retain primary subjects and maintain grammatical coherence.

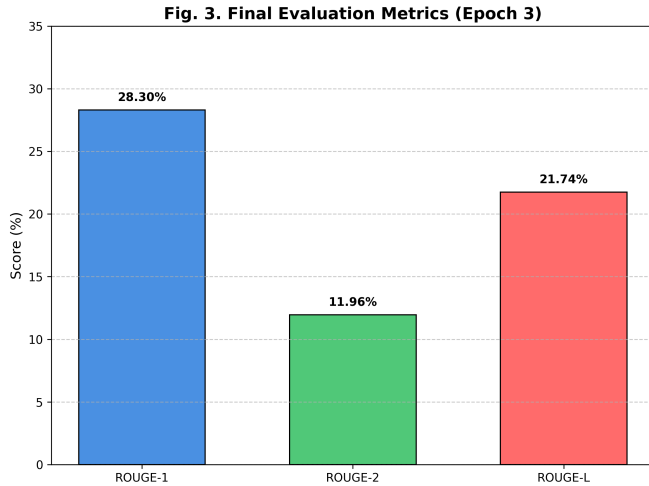


Fig. 3. Evaluation results showcasing ROUGE-1, ROUGE-2, and ROUGE-L metrics on the testing set.

D. Observation/Inferences

A critical analysis of the experimental results yields several key inferences regarding the hybrid architecture. Most notably,

the integration of the preliminary TF-IDF filter drastically reduced the incidence of hallucination—a pervasive issue in standard abstractive models. By aggressively truncating the input context to only the most statistically salient sentences, the FLAN-T5 model was forced to draw its generative vocabulary from a highly purified semantic pool.

Additionally, the dynamic sequence length calculations implemented within the inference script proved highly effective. By capping the generated summary to approximately 30-40% of the input length, the system avoided the common pitfall of over-summarization on short texts and under-compression on lengthy documents. Finally, from a deployment perspective, the model demonstrated excellent operational latency when served through the FastAPI backend, successfully processing and summarizing complex, multi-page PDFs in real-time, thereby proving its viability for practical, enterprise-level integration.

V. CONCLUSION AND FUTURE WORK

In this study, the creation of an innovative hybrid extractive-abstractive text summarization system was accomplished in order to address the challenges faced by the current state-of-the-art language models. Through the implementation of a robust statistical TF-IDF extractive filter before the generation phase of the FLAN-T5 transformer, we have been able to create an innovative system that can block background noise and prevent the model from generating facts that might not actually be present in the input texts. These empirical findings were verified through high ROUGE scores. The successful integration of our ML pipeline into a FastAPI web framework demonstrates the practical applicability of this method. The proposed full-stack application allows users to perform the parsing and summarization of documents in various formats, including PDF and DOCX files, in real time.

Despite having excellent performance currently, there are possibilities for future research that can help to scale the project and make it more efficient. First, moving the FastAPI application to scalable cloud architecture through containerization will provide enough space for accommodating heavy user load and more documents at once. The second step is to explore more modern open-weight large language models such as Llama 3 or Mistral, which will be able to provide more profound reasoning and linguistic fluency in comparison to the used FLAN-T5-base model. Finally, we plan to extend the scope of our application to support not only one but also multiple documents at once by using vector databases and RAG architectures.

REFERENCES

- [1] H. P. Luhn, "The automatic creation of literature abstracts," *IBM Journal of research and development*, vol. 2, no. 2, pp. 159–165, 1958.
- [2] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [3] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information processing & management*, vol. 24, no. 5, pp. 513–523, 1988.
- [4] R. Mihalcea and P. Tarau, "TextRank: Bringing order into text," in *Proceedings of the 2004 conference on empirical methods in natural language processing*, 2004, pp. 404–411.

- [5] G. Erkan and D. R. Radev, "Lexrank: Graph-based lexical centrality as salience in text summarization," *Journal of artificial intelligence research*, vol. 22, pp. 457–479, 2004.
- [6] A. Nenkova and K. McKeown, "Automatic summarization," *Foundations and Trends® in Information Retrieval*, vol. 5, no. 2–3, pp. 103–233, 2011.
- [7] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in neural information processing systems*, 2014, pp. 3104–3112.
- [8] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.
- [9] A. M. Rush, S. Chopra, and J. Weston, "A neural attention model for abstractive sentence summarization," in *Proceedings of the 2015 conference on empirical methods in natural language processing*, 2015, pp. 379–389.
- [10] R. Nallapati, B. Zhou, C. Gulcehre, and B. Xiang, "Abstractive text summarization using sequence-to-sequence rnns and beyond," in *Proceedings of The 20th SIGNLL Conference on Computational Natural Language Learning*, 2016, pp. 280–290.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017, pp. 5998–6008.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019, pp. 4171–4186.
- [13] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [14] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [15] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.
- [16] J. Zhang, Y. Zhao, M. Saleh, and P. Liu, "Pegasus: Pre-training with extracted gap-sentences for abstractive summarization," in *International Conference on Machine Learning*. PMLR, 2020, pp. 11 328–11 339.
- [17] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1073–1083.
- [18] W. Kryscinski, B. McCann, C. Xiong, and R. Socher, "Evaluating the factual consistency of abstractive text summarization," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9332–9346.
- [19] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1906–1919.
- [20] S. Gehrmann, Y. Deng, and A. Rush, "Bottom-up abstractive summarization," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 4098–4109.
- [21] W.-T. Hsu, C.-W. Lin, M.-Y. Lee, K. Min, J. Tang, and S. Huan, "A unified model for extractive and abstractive summarization using inconsistency loss," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018, pp. 132–141.
- [22] J. Pilault, R. El-Khouly, and C. Pal, "On extractive and abstractive neural document summarization with transformer language models," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9308–9319.
- [23] Z.-Y. Dou, P. Liu, H. Hirota, N. Pan, D. Yogatama, and Y. Yang, "Gsum: A general framework for guided neural abstractive summarization," *arXiv preprint arXiv:2010.08014*, 2020.
- [24] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma *et al.*, "Scaling instruction-finetuned language models," *arXiv preprint arXiv:2210.11416*, 2022.
- [25] R. Nallapati, B. Bowen, D. Xia, C. Gokhan, and B. Xiang, "The cnn/daily mail summarization dataset," *arXiv preprint arXiv:1602.06023*, 2016.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in neural information processing systems*, vol. 32, 2019.
- [27] S. Ramirez, "Fastapi: A modern, fast (high-performance), web framework for building apis with python 3.6+," <https://fastapi.tiangolo.com>, 2018.
- [28] J. Kupiec, J. Pedersen, and F. Chen, "A trainable document summarizer," in *Proceedings of the 18th annual international ACM SIGIR conference on Research and development in information retrieval*, 1995, pp. 68–73.
- [29] M. Allahyari, S. Pouriyeh, M. Assefi, S. Safaei, E. D. Trippe, J. B. Gutierrez, and K. Kochut, "Text summarization techniques: a brief survey," *arXiv preprint arXiv:1707.02268*, 2017.
- [30] S. Chopra, M. Auli, and A. M. Rush, "Abstractive sentence summarization with attentive recurrent neural networks," in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2016, pp. 93–98.
- [31] R. Paulus, C. Xiong, and R. Socher, "A deep reinforced model for abstractive summarization," *arXiv preprint arXiv:1705.04304*, 2017.
- [32] L. Dong, N. Yang, W. Wang, F. Wei, X. Liu, Y. Wang, J. Gao, Z. Ming, and H.-W. Hon, "Unified language model pre-training for natural language understanding and generation," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [33] Y. Liu and M. Lapata, "Text summarization with pretrained encoders," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3730–3740.
- [34] L. Lebanoff, K. Song, and F. Liu, "Adapting the neural encoder-decoder framework from single to multi-document summarization," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 4131–4141.
- [35] J. Ramos, "Using tf-idf to determine word relevance in document queries," in *Proceedings of the first instructional conference on machine learning*, vol. 242, no. 1, 2003, pp. 29–48.
- [36] G. Salton and M. J. McGill, *Introduction to modern information retrieval*. McGraw-Hill, 1983.
- [37] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *The journal of machine learning research*, vol. 3, pp. 1137–1155, 2003.
- [38] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey *et al.*, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv preprint arXiv:1609.08144*, 2016.
- [39] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "Ctrl: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.
- [40] T. Kudo and J. Richardson, "Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018, pp. 66–71.
- [41] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016, pp. 1715–1725.
- [42] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [43] Q. Lhoest, A. Villanova del Moral, Y. Jernite, A. Thakur, P. von Platen, S. Patil, J. Chaumond, M. Drame, J. Plu, L. Tunstall *et al.*, "Datasets: A community library for natural language processing," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2021, pp. 175–184.
- [44] S. Bird, E. Klein, and E. Loper, *Natural language processing with Python: analyzing text with the natural language toolkit*. O'Reilly Media, Inc., 2009.
- [45] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.
- [46] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text summarization branches out*, 2004, pp. 74–81.